

NumPy and Matplotlib

EEC 3150

Trevecca Nazarene University

1 NumPy

- Intro to NumPy
- Creating ndarray's
- Indexing ndarray's
- Math operations on ndarray's
- Misc. Array Tools
- Math Functions
- Random Functions
- Statistics Functions
- Linear Algebra Functions

NumPy: Numerical Python

NumPy:

NumPy is one of the most commonly used third-party libraries in Python

<https://numpy.org/doc/stable/>

Key features:

- Extensive library of tools for mathematical computations
 - Trigonometric, exponential, and special functions
 - Statistics and random number generation
 - Linear algebra and Fourier analysis (and more)
- Bread and butter: `ndarray` objects
 - n -dimensional arrays
 - Lists but infinitely better (how you wish lists worked)

Importing numpy:

```
import numpy as np
```

Creating ndarray's

Functions for creating arrays:

- `np.array(x)` will convert the list `x` into an array
The function accepts an additional keyword argument `dtype` that allows for the specification of data type the array should have (ex: `np.int32`, `np.float64`)
- `np.zeros(n)` will create an array of length `n` (int) of all zeros
You can also pass a tuple instead of an integer to create a multi-dimensional array (ex: `np.zeros((m,n))` creates and $m \times n$ array)
- `np.ones(n)` (like above, but all ones)
- `np.full(n,value)` (like above, but fills array with `value`)
- `np.arange(a,b,step)` same as Python `range` but allows floats
- `np.linspace(a,b,n)` creates an array of `n` evenly-spaced points from `a` to `b`
- `np.random.rand(n)` creates a length `n` array of random numbers b/t 0 and 1
This uses the `np.random` library (more on that later)

Some additional points:

- The different dimensions of a numpy array are called *axes* (*axis* is often a keyword argument to many functions)
- Use the `shape` attribute to get the length of an array along each axis (ex: `x.shape`)

Creating ndarray's

2 ways of creating a 3×3 array of all 5's:

- `np.full((3,3),5)`
- `5*np.ones((3,3))`

Row vs. column vs. 1-D:

These are all different objects:

- `np.ones(5)`
This is a 1-D array
- `np.ones((5,1))`
This is a 2-D array (column)
- `np.ones((1,5))`
This is a 2-D array (row)

Indexing ndarray's

Indexing ndarray's

Standard list indexing:

- Index an element: `x[1]`
- Get a slice: `x[4:12]`
- Index from the end: `x[-3]`

Numpy also supports *fancy indexing*:

- Index a 2-D array: `x[2,5]`
- Index an entire row, column respectively: `x[2,:]`, `x[:,5]`
- Index a list of specific elements: `x[[1,4,8]]`
- Index with a boolean mask: `x[[True,False,False,True]]`
- Index all elements with values greater than 0: `x[x>0]`
 - How it works: `x>0` creates a boolean mask used to index the `x` array
 - This works for multi-dimensional arrays too
 - Limitation: you can't do multiple conditionals in the same fancy index
- Get the indices (not values) where an array is greater than 0: `np.where(x>0)`
- Indexing with `np.newaxis` will add another axis to the array

In addition to accessing array elements with the above techniques, you can also SET elements with the above techniques

Indexing ndarray's

Conditional masking vs. `np.where()`

- `x[x>0]` returns the VALUES where `x` is greater than 0
- `np.where(x>0)` returns the INDICES where `x` is greater than 0

`np.where()` on multi-dimensional arrays

- Running a `np.where()` on a 2D (or higher) array will produce a tuple of index arrays
- Each tuple corresponds to the indices of the respective axis
- Indexing the original array with this tuple of index arrays will produce a 1D array

Indexing ndarray's

Set negative elements to 0:

- Generate some random data
- Set all negative values to 0

Index with multiple conditions:

- Generate some random data
- Using several boolean masks, index the values such that $a < x < b$
- There are several ways to do this
 - Use numpy's `np.logical_and(x,y)` function
 - Repeatedly index and overwrite the same array with different boolean masks

Math operations ndarray's

Math operations ndarray's:

Numpy arrays can handle mathematical operations in an intuitive, element-wise way:

- We can perform virtually any mathematical operation on any combination of arrays and scalars (as long as their lengths in each dimension match)
- The exception to this is Boolean operations, but Numpy has functions for these anyway
- We can also apply virtually any of Numpy's mathematical functions on arrays in an element-wise fashion
- Numpy can even add arrays of different sizes in certain contexts

Math operations ndarray's

Plot some random numbers:

- Create an array of random numbers b/t $-a$ and a
- Create and plot a linearly-spaced array of numbers with the above random numbers added as noise

Plot sin and cos:

Plot $\sin(x)$ and $\cos(x)$ from 0 to 2π

Plot positive $\sin(x)$:

- Create x, y arrays for plotting $\sin(x)$ from $0, 4\pi$
- Only plot values where $\sin(x) \geq 0$
- Plot with dots

Misc. Array Tools

- `np.reshape(x, (m,n))` reshapes `x` into a new shape; also, `x.reshape((m,n))`
- `x.flatten()` returns a 1-D reshaping of `x` (doesn't overwrite `x`)
- `np.flipud(x)` (ud: up-down) flips a multi-dim array vertically
- `np.fliplr(x)` (lr: left-right) flips a multi-dim array horizontally
- `np.rot90(x,n)` applies an $n * 90$ degree rotation to the array
- `np.roll(x,n)` applies a cyclic shift by n to the right
- `np.sort(x,axis=-1)` returns a sorted version of `x` (doesn't overwrite `x`)
- `np.argsort(x,axis=-1)` returns the indices of `x` which sort it
- `x.T` transposes a 2D array
- `np.unique(x)` returns a sorted array of all the unique elements of `x`
- `np.append(x,value)` returns an array with `value` appended to `x` (doesn't overwrite `x`)
- `np.insert(x,idx,value)` returns an array with `value` inserted to `x` at index `idx` (doesn't overwrite `x`)
- `np.delete(x,idx)` returns an array with `value` deleted from `x` (doesn't overwrite `x`)

Misc. Array Tools

Indices of all unique values:

- Create a list of random integers
- Get unique values
- For each unique value, create of list of indices where the array has that value
- Remove all zeros from the array

Sort data before plot:

- Create random x data from 0 to 2π
- $y = \sin(x)$
- Plot and see that the data is out of order
- Sort the data so that the plot looks correct (argsort on x and apply to x and y)
- Plot again (comment out old plot)

Math Functions

- `sum`, `prod`, `cumsum`, `cumprod`
- `np.amin(x,axis=None)` return the minimum of an array or minimum along an axis
Also: `amax`, `argmin`, and `argmax`
- `degrees`, `radians` (conversion)
- Trig: `sin`, ... , `arcsin`... , `sinh`, ... , `arcsinh`, ...
- Exp: `exp`, `log`, `log2`, `log10`
- `floor`, `ceil`, `lcm`, `gcd`
- Complex numbers: `x = 1+1j`
 - `x.real`, `x.imag`, `np.conj(x)`, `np.angle(x)`

There are also:

- Set functions: `union`, `intersection`, ...
- Boolean functions: `logical_and`, ... , `all`, `isinf`

Random Functions

Random Functions (`np.random`):

<https://numpy.org/doc/stable/reference/random/legacy.html>

All functions begin with `np.random`.

- `seed(n)` set the seed for RNG (useful for reproducible random sequences)
- `rand(n)` create an array of uniform random numbers b/t 0 and 1 (pass multiple args for multi-dim)
- `randn(n)` create an array of normally-distributed random numbers (pass multiple args for multi-dim)
- `randint(low,high,size)` create an array of uniform random integers numbers b/t low (inc) and high (excl)
- `uniform(low,high,size)` create an array of uniform random numbers b/t low and high
- `normal(loc,scale,size)` create an array of normally-distributed random numbers with mean `loc` and std `scale`
- `multivariate_normal(mean,cov,size)` create an array of normally-distributed random numbers with mean and cov
- Many more distributions...

Random Functions (`np.random`):

<https://numpy.org/doc/stable/reference/random/legacy.html>

All functions begin with `np.random`.

- `shuffle(x)` randomly shuffle `x` in-place (DOES CHANGE `x`!!!)
- `permutation(x)` randomly shuffle `x` (doesn't change `x`)
- `choice(arr, size, replace=True, p=None)` return a random sample (w/ or w/out replacement) or `arr` of size with optional weights `p`

Plot a multi-normal distribution:

- Generate $(n,2)$ array of multi-normal data with some mean and covariance
- Plot the data

Sample from a probability distribution:

Use `np.cumsum` and `np.random.rand` to sample from a PDF of your choice (the `np.random` library has a function for this, but doing it manually is a good exercise)

- Create function f for probabilities
- Create grid of x values (limits and number of points)
- Sample f on grid (array of f values)
- Normalize and cumsum the array
- Generate random numbers and determine which sample they fall into

Sample from a probability distribution:

- Redo previous exercise with `np.random.choice`

Statistics Functions:

<https://numpy.org/doc/stable/reference/routines.statistics.html>

- average, mean, median, std, var
- cov, corrcoef
- quantile, percentile (functionally equivalent, different inputs)
- histogram, histogram2d, histogramdd
- bincount (count the number of occurrences of unique integers)

Correlate some variables:

- Create some linspace-d x data
- Create $y = mx + b$ plus normal noise
- Plot x, y
- Get `corrcoef`
- Vary strength of noise

Linear Algebra Functions

Linear Algebra Functions (`np.linalg`):

<https://numpy.org/doc/stable/reference/routines.linalg.html>

NOTE: some of these use `np` and some use `np.linalg`

- `np.eye(n)` creates and $n \times n$ identity matrix
- `A.T` transposes the matrix `A`
- `np.linalg.norm(x,ord)` returns the norm (magnitude) of a vector `x` (optional argument `ord` for specifying the order of the norm)
- `np.matmul(A,B)` performs the matrix product of two vectors/matrices (there's also `dot`, `inner`, `outer`)
- `np.linalg.det(A)` returns the determinant of a matrix
- `np.linalg.inv(A)` returns the inverse of `A`
- `np.linalg.eig(A)` returns the eigenvalues and eigenvectors of a square matrix (can return complex numbers for non-symmetric matrices)
- `np.linalg.solve(A,b)` returns the solution of the system of linear equations $Ax = b$

A possible import alias:

```
from numpy import linalg as LA
```

Linear Algebra Functions

Solve a system of linear equations:

Do this WITHOUT and WITH `np.linalg.solve`

- Create a system of equations $Ax = b$ where A is some matrix and x, b are some vectors
- Solve the system by finding $x = A^{-1}b$

Find the principal components of some data:

- Generate some random normal x data
- Generate y data that is linear w.r.t. x plus normal noise
- Evaluate correlation matrix (`corrcoef`)
- Evaluate eigenvectors and values of `corrcoef`
- Plot the vectors and the data